



MÓDULO 2 · T07

Listas y Navegación: Tab Navigator

 *Navegación por pestañas*

Carlos Rodríguez Contreras · GitHub: `karrocon`

¿Qué aprenderás hoy?

En esta sesión construirás una navegación por pestañas completa con iconos, aprenderás a anidar navegadores y a controlar la visibilidad de la barra de tabs según el contexto.

1

Tab Navigator con iconos

Crear un Tab Navigator con iconos usando `@react-navigation/bottom-tabs` e `Ionicons`.

2

Stack dentro de Tab

Anidar un Stack Navigator dentro de una pestaña para navegación en profundidad.

3

Ocultar la barra de tabs

Controlar la visibilidad del tab bar con `tabBarStyle: { display: 'none' }` en pantallas específicas.

4

Jerarquía Root Stack

Entender la estructura Root Stack → Tabs → Stack anidado en una app real.

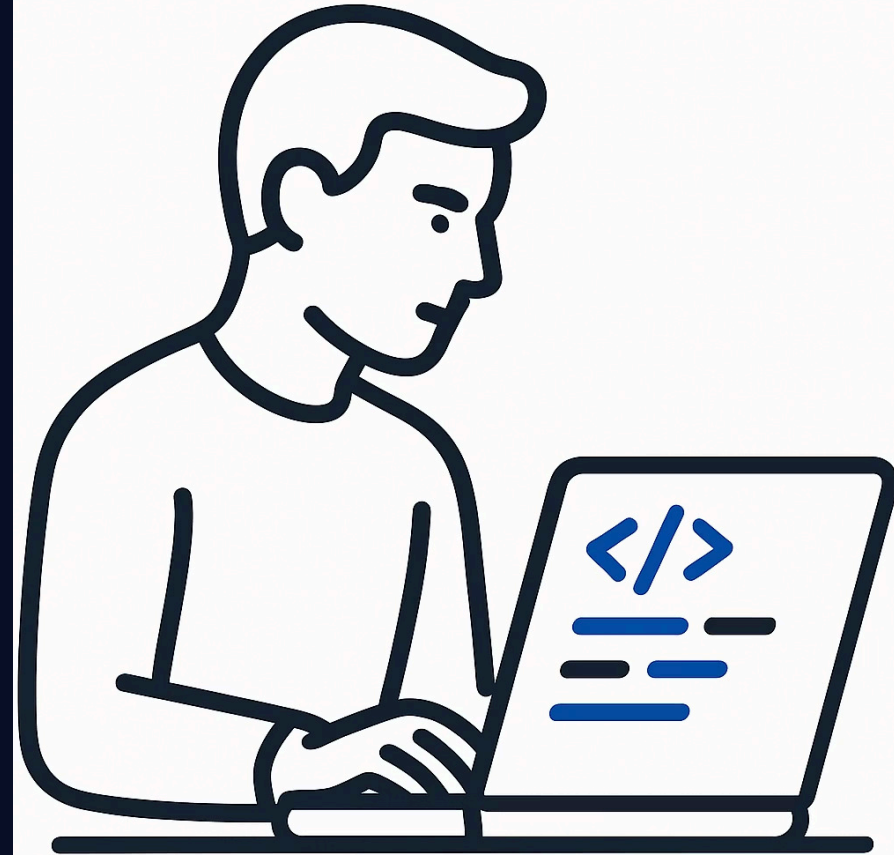
Instalación

Antes de empezar, instala las dependencias necesarias. El paquete `@react-navigation/bottom-tabs` provee el navegador de pestañas, y `@expo/vector-icons` incluye Ionicons para los iconos del tab bar.

```
npm install @react-navigation/bottom-tabs
```

```
npx expo install @expo/vector-icons
```

❗ Asegúrate de tener ya instalados `@react-navigation/native` y sus dependencias peer antes de añadir el navegador de tabs.



Tab básico

¿Cómo funciona?

Se crea un navegador con `createBottomTabNavigator` y se definen las pantallas con `Tabs.Screen`. Cada pantalla obtiene un nombre, un componente y opciones como el título visible.

El parámetro `tabBarActiveTintColor` en `screenOptions` controla el color del tab activo para toda la barra.

- ✓ Tipando el navegador con `TabsParamList` obtienes autocompletado y validación en las rutas.

Código

```
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';

const Tabs = createBottomTabNavigator<TabsParamList>();

function MainTabs() {
  return (
    <Tabs.Navigator screenOptions={{tabBarActiveTintColor: '#EF5350'}}>
      <Tabs.Screen
        name="HomeTab" component={HomeScreen}
        options={{ title: 'Inicio' }}
      />
      <Tabs.Screen
        name="PokedexTab" component={PokedexScreen}
        options={{ title: 'Pokédex' }}
      />
      <Tabs.Screen
        name="AjustesTab" component={AjustesScreen}
        options={{ title: 'Ajustes' }}
      />
    </Tabs.Navigator>
  );
}
```

Añadir iconos con Ionicons

La prop `tabBarIcon` recibe una función con los parámetros `focused`, `color` y `size`. Usando `focused` puedes alternar entre el icono relleno y el de contorno para indicar la pestaña activa.

```
import Ionicons from '@expo/vector-icons/Ionicons';

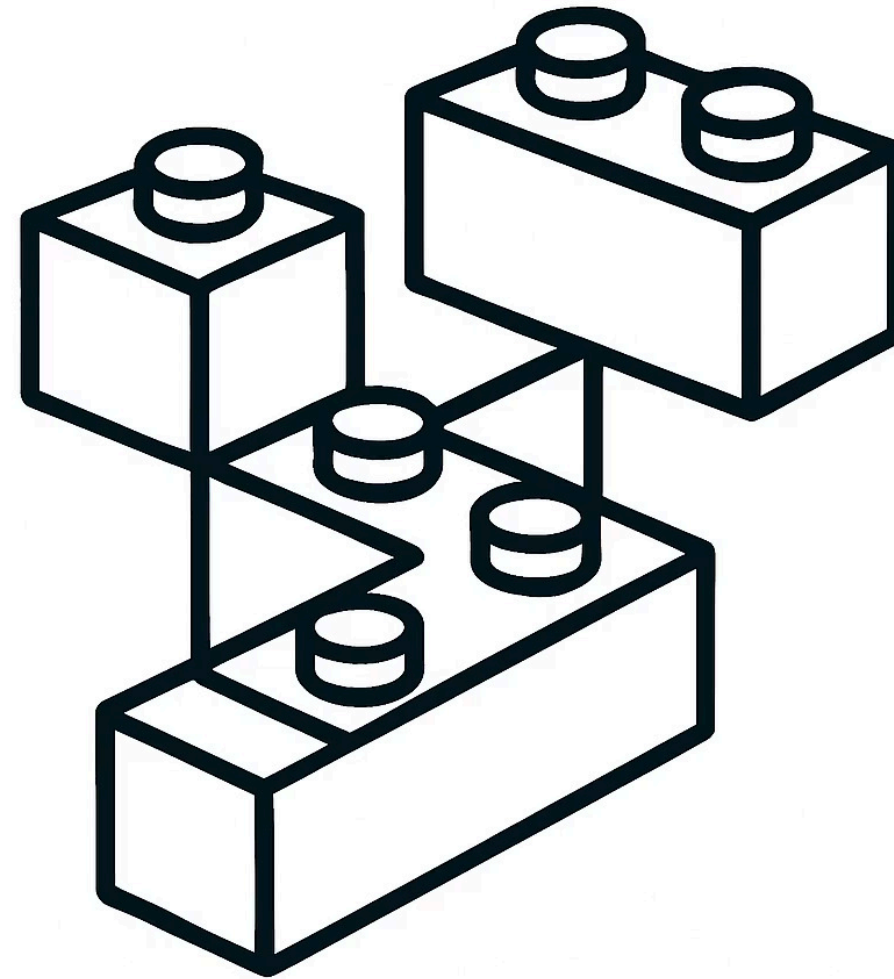
<Tabs.Screen
  name="PokedexTab"
  component={PokedexScreen}
  options={{
    title: 'Pokédex',
    tabBarIcon: ({ focused, color, size }) => (
      <Ionicons
        name={focused ? 'book' : 'book-outline'}
        size={size}
        color={color}
      />
    ),
  }}
/>
```

 `color` y `size` son gestionados automáticamente por el navegador – úsalos directamente en el icono para respetar el tema visual de la barra.

Anidar Stack dentro de Tab

Para navegar en profundidad desde una pestaña (p. ej. de Pokédex al Detalle de un Pokémon), se crea un Stack Navigator propio y se pasa como `component` de la pestaña correspondiente.

```
function PokedexStackNav() {  
  return (  
    <PokedexStack.Navigator>  
      <PokedexStack.Screen  
        name="Pokedex"  
        component={PokedexScreen}  
      />  
      <PokedexStack.Screen  
        name="Detalle"  
        component={DetalleScreen}  
        options={{ route }} => ({  
          title: route.params.nombre  
        })  
      />  
    </PokedexStack.Navigator>  
  );  
}  
  
// En el Tab:  
<Tabs.Screen  
  name="PokedexTab"  
  component={PokedexStackNav}  
  options={{ title: 'Pokédex', ... }}  
/>
```



PokeApp — jerarquía completa

Esta es la estructura de navegación completa de la PokeApp. Cada nivel tiene una responsabilidad clara: el Root Stack gestiona el flujo inicial (Splash → App), los Tabs organizan las secciones principales, y los Stacks anidados permiten la navegación en profundidad.

```
NavigationContainer
├── RootStack
│   ├── Splash
│   └── Main → MainTabs
│       ├── HomeTab → HomeStack
│       │   ├── Home
│       │   ├── ConfigurarEquipo
│       │   └── Batalla
│       ├── PokedexTab → PokedexStack
│       │   ├── Pokedex
│       │   └── Detalle
│       └── AjustesTab → Settings
```

❗ Mantener el Root Stack fuera de los Tabs permite mostrar pantallas modales (como Splash) que no pertenecen a ninguna pestaña en concreto.

Ocultar tabs en pantallas específicas

¿Cuándo ocultarlos?

Hay pantallas que requieren toda la pantalla sin distracciones: la pantalla de Batalla o la de Configurar Equipo no deberían mostrar la barra de tabs. Se usa `getFocusedRouteNameFromRoute` para detectar la ruta activa dentro del Stack anidado.

Si la ruta activa coincide con una pantalla que debe ocultar el tab bar, se aplica `display: 'none'` al `tabBarStyle`.

⚠ `getFocusedRouteNameFromRoute` devuelve `undefined` cuando la ruta no ha navegado todavía – por eso se usa el operador `??` para asignar la pantalla inicial por defecto.

Código

```
<Tabs.Screen
  name={HOME_TAB}
  component={HomeStackNav}
  options={({ route }) => {
    const focused =
      getFocusedRouteNameFromRoute(route)
      ?? HOME;
    const hideBar =
      focused === BATALLA ||
      focused === CONFIGURAR_EQUIPO;
    return {
      tabBarStyle: hideBar
        ? { display: 'none' }
        : {
            backgroundColor:
              theme.backgroundCard
          },
    };
  }}
/>
```


Root Stack → Splash → Tabs

El punto de entrada de la PokeApp envuelve todo en un `ThemeProvider` y un `NavigationContainer`. El Root Stack controla la transición desde la pantalla de Splash hasta el sistema de tabs principal, con `headerShown: false` para ocultar la cabecera en ambas pantallas.

```
// App.tsx de PokeApp
export default function App() {
  return (
    <ThemeProvider>
      <NavigationContainer>
        <RootStack.Navigator
          screenOptions={{ headerShown: false }}
        >
          <RootStack.Screen
            name={SPLASH}
            component={SplashScreen}
          />
          <RootStack.Screen
            name={MAIN}
            component={MainTabs}
          />
        </RootStack.Navigator>
      </NavigationContainer>
    </ThemeProvider>
  );
}
```

❗ Usar `headerShown: false` en el Root Stack evita mostrar una cabecera doble cuando las pantallas anidadas tienen su propio header.

Tour starter/App.tsx

Este es el fichero que encontrarás en la carpeta `starter/`. Contiene un Tab Navigator básico con una sola pestaña y un `TODO` que deberás completar: añadir las pestañas de Pokédex y Ajustes con sus respectivos iconos.

```
const Tab = createBottomTabNavigator<TabsParamList>();

export default function App() {
  return (
    <NavigationContainer>
      <Tab.Navigator>
        <Tab.Screen
          name="Inicio"
          component={InicioScreen}
        />
        { /* TODO: añade Pokédex y Ajustes con iconos */ }
      </Tab.Navigator>
    </NavigationContainer>
  );
}
```

⚠ No modifiques la pestaña de Inicio ya existente – céntrate en añadir las dos pestañas marcadas con el `TODO` e incorpora los iconos con `Ionicons`.

Ejercicio T07

Pon en práctica todo lo aprendido. Abre el proyecto en tu editor y sigue los pasos en orden – cada uno construye sobre el anterior. Los bonuses son opcionales pero muy recomendables para afianzar la navegación anidada y el control del tab bar.

1 Abre el fichero de inicio

Navega a `starter/App.tsx` en tu editor.

2 Añade las 3 pestañas

Completa el Tab Navigator con las pestañas: Inicio, Pokédex y Ajustes.

3 Añade iconos con Ionicons

Incorpora `tabBarIcon` en cada pestaña usando `Ionicons` con variante `focused`.

4 Bonus: anida un Stack en Pokédex

Crea un `PokedexStackNav` y úsalo como componente de la pestaña Pokédex.

5 Bonus: ocultar tab bar en Detalle

Navega al Detalle de un Pokémon y oculta el tab bar usando `tabBarStyle: { display: 'none' }`.

Resumen de la sesión

Estos son los conceptos clave que has trabajado hoy. Dominarlos te permitirá estructurar la navegación de cualquier app React Native con múltiples secciones, rutas anidadas y comportamientos visuales dinámicos.

Concepto	Sintaxis clave	Para qué sirve
<code>createBottomTabNavigator</code>	<code>createBottomTabNavigator()</code>	Crea pestañas inferiores
<code>tabBarIcon</code>	<code>{ focused, color, size } => JSX</code>	Icono personalizado por pestaña
<code>tabBarActiveTintColor</code>	<code>tabBarActiveTintColor: '#EF5350'</code>	Color del tab activo
Stack anidado	Componente dentro de una pantalla de tab	Navegación en profundidad por pestaña
Ocultar tab bar	<code>tabBarStyle: { display: 'none' }</code>	Esconder tabs en pantallas específicas
Root Stack	Envuelve los tabs + pantallas modales	Gestiona el flujo inicial de la app

✅ ¡Enhorabuena! Ahora tienes las herramientas para crear una navegación por pestañas robusta, con stacks anidados y control total sobre la visibilidad del tab bar. 🚀